The `hot_delete_policy` defines a policy with two phases: hot and delete. Here's what the definition states:

- *Hot phase*—The index is expected to live for at least one day before carrying out the actions. The block defined in the `actions` object sets a priority (`250`, in this example). Indexes with a higher priority are considered first during node recovery.
- *Delete phase*—The index is deleted once the hot phase completes all the actions. As there is no `min_age` on the delete phase, the delete action kicks in immediately when the hot phase finishes.

How do we attach this policy to an index? That's what we discuss next.

Now that we have defined the policy, the next step is to associate an index with it. To see this in action, let's create an index and attach the policy from listing 6.48 to it.

Listing 6.49 Creating an index with an associated index lifecycle

```
PUT hot_delete_policy_index
{
  "settings": {
    "index.lifecycle.name":"hot_delete_policy" # Name of the policy
  }
}
```

Uses the settings object to set the property

This script creates the `hot_delete_policy_index` index with a property setting on the index: `index.lifecycle.name`. The index is now associated with a lifecycle policy, as `index.lifecycle.name` points to our previously created policy (`hot_delete_policy`) from listing 6.48. This means the index undergoes a phase transition based on the policy definition. When the index is created, it first enters the hot phase and stays in that phase for a day (`min_age=1d`, as defined in our policy) before it applies a few actions (in this case, setting a priority on the index).

As soon as the hot phase completes (one day, based on the policy definition), the index transitions into the next stage: the delete phase, in this case. This is a straightforward phase where the index is deleted automatically.

> **NOTE** The `hot_delete_policy` policy defined in listing 6.48 deletes the index after one day based on the definition. Be aware that if you use this policy in production, you may find no index available after the delete phase (the delete phase purges everything).

To sum up, attaching an index lifecycle policy to an index transitions the index into given phases and executes certain actions defined in each phase. We could define an elaborate policy such as a hot phase for 45 days, a warm phase for one month, and then a cold phase for a year, and finally delete the index after a year.

Suppose we want to transition indexes based on conditions, such as every month or at a particular size. Fortunately, Elasticsearch provides automated and conditional index lifecycle rollovers.